E4 Educator v2.0

Quick Start Guide

Walark Electronics · Fundrum Engineering

Windows 10 / 11 · May 2026

Welcome to the E4 Educator

Congratulations on getting your hands on the Enceladus Educator v2.0. You're about to start a journey from first firmware flash through to building real, useful connected devices — and we're going to enjoy the ride.

This guide gets you from unboxing to your first working program in under 30 minutes. Take your time, read each step before you do it, and don't skip steps — they all matter.

1 What's in the box

- E4 Educator v2.0 board (Enceladus Educator)
- ESP32 DevKit V1 module — pre-fitted in rear sockets
- 2.8" ILI9341 TFT display module — socketed
- 0.96" SSD1306 OLED display module — socketed
- USB-C cable
- This Quick Start Guide

Good to know
The ESP32 module is fitted in sockets — it can be lifted out and swapped between projects. This is a deliberate design feature you'll use throughout the course.

2 Board tour

Before plugging anything in, take a minute to get familiar with the board layout.

Location	What's there	What it does
Top centre	USB-C connector	Power and programming — connect your cable here
Upper half	2.8" TFT display	Your main colour display — 240×320, touch-enabled, SD card on rear
Below TFT left	OLED display	Entry-level 128×64 display — where you start learning about screens
Bottom edge	BOOT RST LEDs NEXT ENTER	Your main user controls — you'll use these constantly
Bottom row	OLED UART I2C-2 AHT20 headers	Sensor and expansion connectors
Right edge	Test points	Diagnostic points for a logic analyser or oscilloscope
Top left/right	MIC and MEMS headers	Analog and digital microphone connections
Rear	ESP32 DevKit V1	The brain — socketed so you can swap it between projects

⚠ Important — 3V3 devices only
Every connector and header on this board runs at 3.3V. Never connect a 5V sensor or module directly — you will damage the ESP32. Always check your sensor's voltage rating first.

3 Install VS Code and PlatformIO

PlatformIO is the development environment you'll use for every project in this course. It runs inside Visual Studio Code (VS Code) — Microsoft's free code editor.

3.1 Install VS Code

1. Open your browser and go to: code.visualstudio.com
2. Click Download for Windows
3. Run the installer — accept the defaults, but tick "Add to PATH" when offered
4. Launch VS Code when the installer finishes

3.2 Install PlatformIO

5. In VS Code, click the Extensions icon in the left sidebar (four squares)
6. In the search box type: PlatformIO IDE
7. Click Install on the PlatformIO IDE extension by PlatformIO
8. Wait for the installation to complete — this takes a few minutes the first time
9. When prompted, restart VS Code

Tip
The PlatformIO install downloads tools in the background. You'll see a loading indicator in the bottom status bar. Wait until it stops before continuing — trying to create a project too early can cause confusing errors.

4 Install the USB driver

The ESP32 DevKit V1 uses a CP2102 USB-to-serial chip. Windows needs a driver to talk to it. If you've used an ESP32 before, you may already have this.

10. Go to: silabs.com/developers/usb-to-uart-bridge-vcp-drivers
11. Download the CP210x Windows Drivers
12. Extract the zip and run CP210xVCPInstaller_x64.exe
13. Follow the installer — click Next and Finish
14. Plug the E4 board into your PC using the USB-C cable
15. Open Windows Device Manager (right-click Start → Device Manager)
16. Look for Ports (COM & LPT) → you should see Silicon Labs CP210x USB to UART Bridge (COMxx)
17. Note the COM port number — you'll need it shortly

⚠ If you don't see the COM port
Try a different USB-C cable — many cables are charge-only and carry no data. A data-capable cable is essential. If you still don't see it, try a different USB port on your PC.

5 Create your first project

Time to create your first PlatformIO project. This is the structure every E4 project will follow.

5.1 Create the project

18. In VS Code, click the PlatformIO icon in the left sidebar (the ant head)
19. Click New Project
20. Fill in the project wizard:

Field	Value
Name	`E4_HelloWorld`
Board	Espressif ESP32 Dev Module
Framework	Arduino
Location	Leave as default or choose your projects folder

21. Click Finish and wait while PlatformIO sets up the project

5.2 Configure platformio.ini

The platformio.ini file controls everything about how your project builds and uploads. Open it from the project root and replace the contents with the E4 v2.0 standard configuration.

Find your COM port number from Step 4 and replace COM10 in the template below:

```
[env:e4_educator_v2]
platform   = espressif32@6.6.0
board      = esp32dev
framework  = arduino
monitor_speed = 115200
monitor_port  = COM10
upload_port   = COM10

build_flags =
  -DFW_VERSION=\"1.0.0\"
  -DFW_DATE=\"2026-05-01\"
  -DLED_RED=16 -DLED_GREEN=26 -DLED_BLUE=32
  -DBTN_NEXT=13 -DBTN_ENT=14
  -DTFT_CS=5 -DTFT_DC=17 -DTFT_BL=12
  -DTFT_MOSI=23 -DTFT_SCLK=18 -DTFT_MISO=19
  -DTOUCH_CS=33 -DSD_CS=15
  -DI2C_SDA=21 -DI2C_SCL=22
  -DONE_WIRE=4 -DLDR=39 -DMIC_ANALOG=36
  -DMEMS_BCLK=32 -DMEMS_WS=35 -DMEMS_SD=34
  -DDAC_AUDIO=25 -DPIEZO=27
```

Why do we pin the platform version?

The line platform = espressif32@6.6.0 locks the compiler and tools to a specific known-good version. Without this, PlatformIO might update the platform automatically and introduce subtle breaking changes. Always pin the version in every E4 project.

6 Your first program — blink the LED

Every embedded systems journey begins with blinking an LED. Open src/main.cpp and replace the contents with this:

```
#include <Arduino.h>

void setup() {
  Serial.begin(115200);
  Serial.println("E4 Educator v2.0 — Hello World!");

  pinMode(LED_RED,   OUTPUT);
  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_BLUE,  OUTPUT);

  // Start with all LEDs off
  digitalWrite(LED_RED,   LOW);
  digitalWrite(LED_GREEN, LOW);
  digitalWrite(LED_BLUE,  LOW);
}

void loop() {
  // Cycle through Red, Green, Blue
  digitalWrite(LED_RED, HIGH);
  delay(500);
  digitalWrite(LED_RED, LOW);

  digitalWrite(LED_GREEN, HIGH);
  delay(500);
  digitalWrite(LED_GREEN, LOW);

  digitalWrite(LED_BLUE, HIGH);
  delay(500);
  digitalWrite(LED_BLUE, LOW);
}
```

Notice something?
We're using LED_RED, LED_GREEN and LED_BLUE — not raw GPIO numbers. These names come from the build_flags in platformio.ini. This is the Fundrum convention: GPIO numbers live in one place only, never scattered through your code.

7 Build and upload

22. Make sure the E4 board is connected via USB-C
23. In VS Code, press Ctrl+Alt+U to build and upload (or click the → arrow in the bottom toolbar)
24. Watch the terminal at the bottom — you'll see the code compile, then upload
25. The board resets automatically after upload
26. Your LEDs should now cycle Red → Green → Blue

To see the Serial message, open the Serial Monitor with Ctrl+Alt+S. You should see:

```
E4 Educator v2.0 — Hello World!
```

If the upload fails

Check the COM port number in platformio.ini matches what Device Manager showed. If you see "Connecting..." and dots for a long time, hold the BOOT button on the board, tap RESET, then release BOOT. This forces the ESP32 into flash mode.

8 What's next

You've successfully set up PlatformIO, configured your E4 board, and run your first firmware. That's a genuinely good start.

The E4 Educator course continues with:

- T01 — PlatformIO and the E4 in depth (millis() timing, non-blocking code, Serial debugging)
- T02 — GPIO and build_flags (understanding the Fundrum pin convention)
- T03 — Buttons and debouncing (making your NEXT and ENTER buttons reliable)
- T04 — I2C fundamentals (reading your first sensor)
- T05 — OLED display (your first screen project)

A note on delay()

You'll notice the Hello World program uses delay(). This is fine for a first program, but it's a habit to break early. T01 shows you why delay() causes problems and how to replace it with millis() — a small change that makes a huge difference to what your firmware can do.